

Mind uploading through machine learning approximation of single neurons: structure and computational cost

Chaya2766

last updated January 6, 2026

Abstract

Calculations of the computational resources required to create and run a human brain emulation based on the paper "Single cortical neurons as deep artificial neural networks" by David Beniaguev, Idan Segev & Michael London.

I present a way to emulate a human brain based on my interpretation of the paper, and calculate the costs of both creating and running such a brain emulation. Some additional suggestions as to how such a system would work are made based on my own understanding of human brain function.

The question of how the necessary data would be gathered from a living brain remains untouched as a subject for exploration at another time.

DISCLAIMER AND WARNING

This is not a scientific paper or engineering advice. This document is only intended as creative world-building and has not been peer-reviewed or approved for any practical application. Use of these ideas in real-world designs or other serious application is entirely at the reader's discretion and risk.

Contents

1	Mimic neurons	2
2	To train a mimic neuron	4
3	Human brain on mimic neurons	5
4	Memory problem in mimic neuron brain emulation	7
5	Summary of computational costs	8
6	OpenSCAD code for illustrations	9

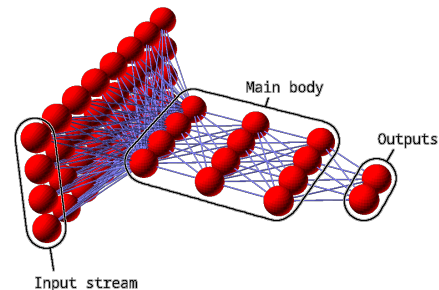
1 Mimic neurons

I need to introduce a new piece of vocabulary to make the explanations less convoluted: a **mimic neuron** will be used throughout this document to refer to a neural network trained to mimic the spiking behaviour of a natural human neuron.

The paper "Single cortical neurons as deep artificial neural networks" by David Beniaguev, Idan Segev & Michael London describes how the previously used models of biological neurons could be replaced with less computationally expensive artificial neural networks. They managed to train artificial neural networks to mimic the activity of a conventionally modeled biological neuron with more than 99% accuracy.

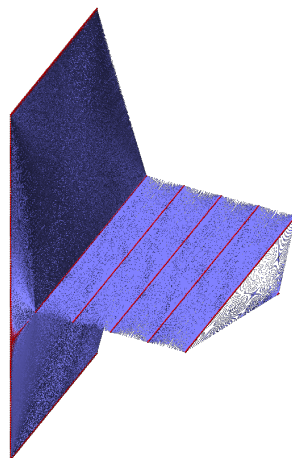
The artificial neural networks in the paper start with neurons which each receive 100 streams of inputs representing input to the neuron's synapses. From this a deep neural network follows, arranged in layers, until two artificial neurons at the end, of which one returns voltage within the neuron and the other the spikes, in timesteps of 1 millisecond.

I am not entirely sure of my interpretation, but as I understand it, the structure of such a mimic neuron could be visualized like this, where the example illustration on the right would show a mimic neuron with 8 input streams each having 4 milliseconds of history, and a main body with 3 layers of 4 neurons each.



In the study, a network with 5 layers of 128 neurons, which takes into account 205 milliseconds of input (among a number of other networks) was found to accurately model an L5PC biological neuron with AMPA & NMDA synapses, which from not being an expert to know exactly what it means I just treat as "a lot of internal complexity".

Just for visualization sake, such a mimic neuron with 100 input streams, 205 milliseconds of history in each input stream, and a main body consisting of 5 layers each having 128 neurons, would look like this:



I will include the code I used to make these visualizations at the end of the document.

Given the structure that I assumed the mimic neuron would have, a simple formula can be made for the number of parameters in the mimic neuron:

$$N_w = (S \times H \times W) + (W^2 \times (D - 1)) + 2W$$

Where:

- N_w is the total Number of Weights in the network
- S is the number of input Streams which the input neurons receive
- H is the history, ie. the length of each input stream
- W is the width of the network, ie. number of neurons in one layer
- D is the depth of the network, ie. number of layers in the main body

$(S \times H \times W)$ is the number of connections between all the input streams and all the neurons in the first layer of the main body, $(W^2 \times (D - 1))$ is the total number of connections across all the layers in the main body, and $2W$ is the number of connections between the last layer of the main body and the two output neurons.

The study presented several networks (their depth, width & history) which reached past the 99% accuracy threshold. I am setting the number of input streams to 7000, as that appears to be the number of synapses that the average human neuron has ([source](#)).

Given that assumption, the number of parameters in these networks would be:

number	Depth	Width	history	accuracy	num. of weights
1	5	128	205ms	99.13%	183 778 560
2	7	128	153ms	99.11%	137 186 560
3	8	256	97ms	99.14%	174 217 728
4	8	256	161ms	99.24%	288 905 728

There appears to be a pattern of more computationally expensive networks being more accurate, though the jump from 99.11% to 99.24% accuracy brought more than a factor of 2 increase in computational cost, so the benefits are diminishing, at least based on this sample.

The obvious choice is network 2 if wanting to minimize computational costs.

Each of the 137 186 560 parameters in the mimic neuron correspond to a multiply-and-accumulate (MAC) operation which has to be performed to fully calculate its response. The simulation is quantized in 1 millisecond intervals, which means that the neuron would require **137 186 560 000 MAC/s** to be simulated at realtime speed. Some hardware can efficiently perform MAC operations, but for more standard computing it would otherwise be two floating point operations, a multiplication and an addition, so it translates to **274 373 120 000 FLOPS**. Alternatively for a more down to physics metric, assuming that a 16-bit MAC operation requires about 2000 bit flips in total, the required bit flip rate would be **274 373 120 000 000 bits/s**

Assuming 16-bit weights, the mimic neuron would require **274.37312 MB** to store.
Here's a summary:

Mimic neuron operational characteristics	
accuracy to real human neuron	99.11%
number of parameters:	137 186 560
computation requirement:	137 186 560 000 MAC/s 274 373 120 000 FLOPS 274 373 120 000 000 bits/s
size on disk or in memory:	274.37312 MB

2 To train a mimic neuron

(DNN = Deep Neural Network — the DNNs mentioned here are the same as the "mimic neurons" I had been talking about)

The paper speaks about the training: "we trained many networks with different hyperparameters and trained each network for 2-14 days on a GPU cluster consisting of several V100, K80 and 2080Ti Nvidia GPUs", and "The total amount of single GPU years needed to fit all DNNs throughout the entire study was 3.4 years", as well as that "we trained a total of 137 DNNs" (though in the paper as far as I can tell this referred only to a specific part of the experiment, so the total might have been more, but that was the only exact count I found).

3.4 years is 107 295 840 seconds. Divided by the 137 trained DNNs that gives 783 183 seconds per DNN.

Out of the GPUs mentioned, the V100 seems to be most powerful ([link to spec sheet](#)) and is capable of 7 to 8.2 TeraFLOPS when using double precision floating point variables, which I assume was used. This provides an estimate of how much computing power would have been needed per DNN:

$$783183s \cdot 8.2TFLOPS = 6.4221006ExaFLOPS \cdot s$$

Roughly $6.4221 \cdot 10^{18}$ floating point operations to train a single DNN. However, the DNNs described would have represented neurons with only 100 synapses, whereas an average real human neuron has 7000 synapses. I am simply multiplying by 70 to close that gap, which yields $4.4955 \cdot 10^{20}$ floating point operations.

As such to create one mimic neuron, total computation needed is **4.4955E+20 floating point operations**, which given the previous assumptions would be equivalent to **2.24775E+20 MAC operations**, and assuming 2000bit flips per MAC or 1000 per floating point operation, **4.4955E+23 bit flips**.

Mimic neuron training costs	
Training cost:	2.24775E+20 MAC operations
	4.4955E+20 floating point operations
	4.4955E+23 bit flips

3 Human brain on mimic neurons

There are about 86 billion neurons in the human brain. A simple multiplication of the mimic neuron's requirements by that many neurons would yield these numbers:

Human brain emulation operational characteristics		
number of parameters:	11 798 044 160 000 000 000	$\approx 1.1798E+19$
computation requirement:	11 798 044 160 000 000 000 000 MAC/s	$\approx 1.798E+22$ MAC/s
	23 596 088 320 000 000 000 000 FLOPS	$\approx 2.360E+22$ FLOPS
	23 596 088 320 000 000 000 000 bits/s	$\approx 2.360E+25$ bits/s
size on disk or in memory:	23.59608832 EB	$\approx 1.888E+20$ bits

Given those numbers, I've gone ahead and calculated the amount of hardware and power required to run the emulation based on a number of devices whose parameters I was able to find:

- AMD Alveo V70 AI accelerator card : 2.693teraFLOPS per watt , 202teraFLOPS per device
- Halio-8 AI Accelerator : 10.4TOPS per watt , 26TOPS per device
(what kind of operation though? I'll just assume FLOPS for now)
- Epiphany V 1024-core RISC processor : 75gigaFLOPS per watt , ??? FLOPS per device
- JUPITER supercomputer, top of Green500 list for june 2024 :
72.73gigaFLOPS per watt , 4.5petaFLOPS per device
- Frontier supercomputer, top of TOP500 list for june 2024 :
52.93gigaFLOPS per watt , 1.206exaFLOPS per device
- PTC, The photonic tensor core described in a paper "Photonic Tensor Cores for Machine Learning" by Mario Miscuglio & Volker J. Sorger : 16 petaFLOPS per device, 8 petaFLOPS per watt
note: it uses 4-bit operations rather than my assumed 16-bit
- Nvidia H100 ([link to datasheet](#)) - 4.32286 teraFLOPS per watt, 1513 teraFLOPS (FP16) per device
- [Lightmatter experimental card](#): 65.5TeraFLOPS at 78W $\approx$ 839.7436GigaFLOPS per watt
- Orion's Arm [Ultimate Chip](#) (fictional): 5.09911 E21 bits/s per device at 100W $\approx$ 1.96112655 E-20 J/bit
- Landauer's limit - $9.569929617E - 24J/K \times Temperature$ per bit flip
 $\approx 2.871E-21J/bit$ @ room temp, $2.613E-23J/bit$ @ cosmic microwave background temp.

device	wattage	amount needed
Alveo V70	8.768GW	116 812 318.4
Halio-8	2.269GW	907 541 858.5
Epiphany V	314.6GW	?
JUPITER	324.4GW	5 243 575.2
Frontier	445.8GW	19 565.6
PTC	2.95MW	1 474 755.5
Nvidia H100	5.458GW	271 852
Lightmatter experimental card	28.1GW	360 245 623.2
Ultimate Chip (fictional)	462.75kW	4627.5
Landauer's limit (room temp 300K)	67.744kW	-
Landauer's limit (cosmic background 2.73K)	616.47W	-

Similarly, simply multiplying the cost of training a single mimic neuron by the 86 billion neurons expected in the human brain yields these training costs:

Human brain emulation training costs	
Training cost:	1.933065E+31 MAC operations 3.86613E+31 floating point operations 3.86613E+34 bit flips

And I've also gone ahead and calculated how much time and energy would be needed to train all the mimic neurons on the devices from the list presented on the previous page.

The time assumes a single device is used for the training, which could be decreased by dividing it across multiple devices, but the total energy should in principle remain the same regardless of whether one or multiple devices are used.

device	total energy	time
Alveo V70	1.436E+19J	1.914E+17s
Halio-8	3.717E+18J	1.487E+18s
Epiphany V	5.155E+20J	?
JUPITER	5.316E+20J	8.591E+15s
Frontier	7.304E+20J	3.206E+13s
PTC	4.833E+15J	2.416E+15s
Nvidia H100	8.943E+18J	2.555E+16s
Lightmatter experimental card	4.604E+19J	5.902E+17s
Ultimate Chip (fictional)	7.582E+14J	7.582E+12s
Landauer's limit (room temp 300K)	1.110E+14J	-
Landauer's limit (cosmic background 2.73K)	1.010E+12J	-

4 Memory problem in mimic neuron brain emulation

Important vocabulary piece:

Engram - a group of neurons whose connections with each other became strengthened such that firing of one or more of them has a significant chance to cause the whole group to fire. Engrams are thought to be the fundamental building block of memories, as their manipulation was shown to directly affect the ability of rats to recall events or associate stimuli.

Neurons naturally form engrams based on their own state and activity as well as the activity of nearby neurons. A neuron shows distinct physiological activity when it is receptive to becoming a part of a new engram, as illustrated by an experiment where the neurons in a rat's brain were genetically modified to produce a specific marker during engram formation.

The model needs to be dynamic, but the mimic neurons are static

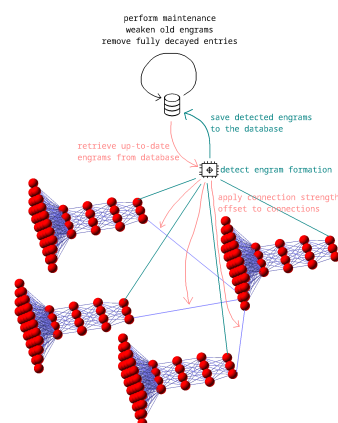
A healthy human brain is constantly rewiring itself, but the mimic neurons have no ability to grow new connections, and in a brain made from them they will never die, nor will new ones be born without intervention. This concern would be surpassed if two assumptions turn out to be true:

1. the human brain can form memories even when restricted to only modifying the strength of existing connections, but not creating new ones
2. the mimic neurons can be trained to respond accurately to these changes in connection strength

Assumption 1 could be wrong completely, but I would expect that at the very least the flexibility of such a brain would be stunted, such as a complete inability to acquire new skills. One plausible workaround would be to rely on assumption 2 and attempt to substitute creation of new connections by adding even more connections right from the start, and adjusting them as needed.

Assumption 2 is not trivial but much more believable, but it would require data of the neuron's activity spanning a long period of time, such that it captures how its function changed in response to the strengthening and weakening of connections. Another issue here is knowing how the connections should be strengthened, and the workaround I see is adding a third output to the mimic neuron, which signals whether or not the neuron is currently receptive to becoming part of an engram. This third output would be monitored by an external mechanism that somehow detects when nearby mimic neurons should form an engram and strengthens connections between them.

I imagine that a monitoring system added to the simulation could detect and calculate such engrams, then store them in a simple SQL database as the IDs of the neurons and the amount of adjustment to the connections between them. The database could be periodically processed to decrease the strength in these entries based on their age, which would provide a rough approximation of forgetting, which is likely also necessary for normal brain function, and eventually remove the entries where the strength dropped down to 0. Another mechanism would periodically go through the database and apply the connection strength modifiers to the brain emulation, keeping it consistent with the database.



This would mean that spiking outputs of mimic neurons are not just copied along the connections, but the output is multiplied, effectively making each such connection another parameter in the system for which a multiply-and-accumulate operation has to be performed on every inference. Additionally the engram detection would likely add some overhead but at the moment I don't know how to speculate about the exact amount of computation it would add to the system.

5 Summary of computational costs

Mimic neuron	
accuracy to real human neuron	99.11%
number of parameters:	137 186 560
computation requirement:	137 186 560 000 MAC/s 274 373 120 000 FLOPS 274 373 120 000 000 bits/s
size on disk or in memory:	274.37312 MB
Training cost:	2.24775E+20 MAC operations 4.4955E+20 floating point operations 4.4955E+23 bit flips

Human brain emulation on mimic neurons		
number of parameters:	11 798 044 160 000 000 000	$\approx 1.1798E+19$
real-time simulation	11 798 044 160 000 000 000 000 MAC/s	$\approx 1.798E+22$ MAC/s
computation requirement:	23 596 088 320 000 000 000 000 FLOPS 23 596 088 320 000 000 000 000 bits/s	$\approx 2.360E+22$ FLOPS $\approx 2.360E+25$ bits/s
size on disk or in memory:	23.59608832 EB	$\approx 1.888E+20$ bits
Total training cost:	1.933065E+31 MAC operations 3.86613E+31 floating point operations 3.86613E+34 bit flips	

6 OpenSCAD code for illustrations

This should all go into one file. The software needed is OpenSCAD, an open source cad program you can download for free, which will compile the code into a 3d model, assuming you put it together correctly.

By default the code produces the example neuron, you can edit parameters in the aneuron function near the bottom of the file (not inside the modules, that will likely just break it all) but do not go too wild on them or your computer might freeze, this code will produce literal millions of individual objects to render if you try to visualize a network that has the actual numbers of neurons and layers that a real one would have.

Sorry for the code so awkwardly split across pages, but it should be easy enough to copy into a single text file. Use search and replace to turn all the "visible space characters" into actual spaces, it should be easy enough to put together, I've decided against splitting the code manually as I could not do so without also making it unclear how the two code blocks should be combined into one file, and no matter how I tried I also couldn't get it to fit onto a single page.

```
neuron_color=[1,0,0];
synapse_color=[0.5,0.5,1];

module bilayer(num_first,num_second,draw_first=true,sep_neurons=5,sep_layers=20,neuron_size=2,synapse_size=0.25){

    //connections first so that neurons are drawn over them rather than under
    for(a_u[1:1:num_first]){
        a_offset_u[
            0,
            (a_u*sep_neurons)+((-1-num_first)_u*sep_neurons/2),
            0
        ];
        for(b_u[1:1:num_second]){
            b_offset_u[
                sep_layers,
                (b_u*sep_neurons)+((-1-num_second)_u*sep_neurons/2),
                0
            ];
            color(synapse_color){
                hull(){
                    translate(a_offset){sphere(synapse_size);}
                    translate(b_offset){sphere(synapse_size);}
                }
            }
        }
    }

    if(draw_first){
        for(a_u[1:1:num_first]){
            a_offset_u[
                0,
                (a_u*sep_neurons)+((-1-num_first)_u*sep_neurons/2),
                0
            ];
            color(neuron_color){
                translate(a_offset){sphere(neuron_size);}
            }
        }
        for(b_u[1:1:num_second]){
            b_offset_u[
                sep_layers,
                (b_u*sep_neurons)+((-1-num_second)_u*sep_neurons/2),
                0
            ];
            color(neuron_color){
                translate(b_offset){sphere(neuron_size);}
            }
        }
    }
}

module multilayer(layers=[5,5,3],sep_neurons=5,sep_layers=20,neuron_size=2,synapse_size=0.25){
    num_layers_u=len(layers);
    for(i_u[num_layers-1:-1:2]){
        layer_offset_u=(i-1)*sep_layers;
        translate(layer_offset,0,0){
            bilayer(layers[i-1],layers[i],false,sep_neurons,sep_layers,neuron_size,synapse_size);
        }
    }
    bilayer(layers[0],layers[1],true,sep_neurons,sep_layers,neuron_size,synapse_size);
}

module aneuron(history=8,instreams=5,layers=[5,5,3],sep_neurons=5,sep_layers=20,neuron_size=2,synapse_size=0.25){
    num_layers_u=len(layers);
    for(i_u[num_layers-1:-1:1]){
        layer_offset_u=(i-1)*sep_layers;
        translate(layer_offset,0,0){
            bilayer(layers[i-1],layers[i],false,sep_neurons,sep_layers,neuron_size,synapse_size);
        }
    }
    //draw neurons
    for(h_u[1:1:history]){
        for(s_u[1:1:instreams]){
            stream_offset_u[
                sep_layers,
                (s_u*sep_neurons)+((-1-instreams)_u*sep_neurons/2),

```

```

            (h_u*sep_neurons)+((-1-history)_u*sep_neurons/2)
            );
            translate(stream_offset){rotate([90,0,0]){
            color(neuron_color){sphere(neuron_size);}
            }
        }
        for(i_u=1:1:layers[0]){
            input_offset=u[
            0,
            (i_u*sep_neurons)+((-1-layers[0])_u*sep_neurons/2),
            0
            ];
            translate(input_offset){
            color(neuron_color){sphere(neuron_size);}
            }
        }
        //draw connections between neurons
        for(i_u=1:1:layers[0]){
            input_offset=u[
            0,
            (i_u*sep_neurons)+((-1-layers[0])_u*sep_neurons/2),
            0
            ];
            for(h_u=1:1:history){
                for(s_u=1:1:instreams){
                    stream_offset=u[
                    -sep_layers,
                    (s_u*sep_neurons)+((-1-instreams)_u*sep_neurons/2),
                    (h_u*sep_neurons)+((-1-history)_u*sep_neurons/2)
                    ];
                    color(synapse_color){hull(){
                    translate(stream_offset){sphere(synapse_size);}
                    translate(input_offset){sphere(synapse_size);}
                    }
                }
            }
        }
    }
}

//example illustration mimic neuron
aneuron(4,8,[4,4,2],sep_layers=100,neuron_size=20,synapse_size=1,sep_neurons=50);

//mimic neuron using the real history, stream and layer numbers
//aneuron(205,100,[128,128,128,128,2],sep_layers=1000,neuron_size=20,synapse_size=1,sep_neurons=50);
//WARNING this requires a lot of memory, I recommend at least 64GB of memory total (ie. sum of your RAM and swap memory). The CSG tree for this model has 2710934 elements, which surpasses the default limit for automatically canceling rendering. If you uncomment it while viewing the model directly in OpenSCAD, it will freeze the program.
//If you want to produce an image from this, my recommended way of doing that is to use the command line:
//openscad_mimic_neuron.scad -o mimic_neuron.png --imgsize=1000,1000 --colorscheme=Starnight --viewall --autocenter --projection=o --csglimit=6000000
//using this method, the rendering lasted a little over 4 hours for me (likely because most of the memory was swap space on my SSD rather than actual RAM) but actually succeeded, producing the image you saw in the document (except that I manually removed the black background)

```